

GraphMinds: Contextual Graph Construction and Analysis for Perspective Identification

Anonymous Author(s)

Abstract

In this paper, we introduce GraphMinds, a novel framework for contextual, graph-based indexing of documents, inspired by GraphRAG. GraphMinds addresses the limitations of proximity-based approaches by capturing indirect semantic relationships that are often missed when relying solely on similarity in embeddings. By identifying latent connections through shared intermediate nodes, GraphMinds uncovers clusters of terms even when they lack direct links. Assuming that communities of nodes identified by GraphMinds are analogous to topics, we use this framework to compare its performance with Latent Dirichlet Allocation (LDA), a conventional topic modelling method. While LDA is widely used, it suffers from stability issues, particularly with an increasing number of topics, and relies on manual intervention for labelling and validation. In contrast, GraphMinds automatically labels 'topics' based on highly connected nodes within communities. By analysing a set of documents on a specific subject, we demonstrate that LDA tends to isolate topics within individual documents, whereas GraphMinds produces diverse clusters that enable meaningful cross-document comparisons by linking seemingly unrelated concepts. Finally, we explore how GraphMinds can contribute to perspective identification by revealing indirect connections across disparate concepts.

CCS Concepts

• **Information systems** → **Graph-based database models**; • **Computing methodologies** → **Knowledge representation and reasoning**.

Keywords

Retrieval-Augmented Generation, Knowledge Graphs, Graph Neural Networks, Large Language Models, Perspective Identification, Topic Spotting

ACM Reference Format:

Anonymous Author(s). 2025. GraphMinds: Contextual Graph Construction and Analysis for Perspective Identification. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, Padua, Italy, July 13–18, 2025*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACM SIGIR'25, Padua, Italy

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

In this project, we compare documents based on the perspectives of their authors. Perspective identification focuses on determining an author's point of view [11] and is traditionally related to stance detection [9]. Recently, researchers have combined stance detection with topic modelling to capture the semantics underlying tweet classification, using the Biterm Topic Model for small texts [6]. However, larger documents, such as the academic papers analysed in this study, require a different approach, such as Latent Dirichlet Allocation (LDA). LDA is a generative model that learns topic distributions and word probabilities. Despite its widespread use, LDA can suffer from consistency issues and often requires subjective adjustments by analysts. Consequently, alternative approaches using Language Models are being explored for topic identification.

Pre-trained Language Models (PLMs) have demonstrated significant advances in knowledge integration tasks. Knowledge-Enhanced PLMs (KE-PLMs) incorporate structured sources, such as knowledge graphs (e.g., KnowBERT and ERNIE), to address the limitations of traditional PLMs [18]. However, integrating heterogeneous knowledge remains challenging, particularly in tasks requiring global comprehension, where indirect relationships often hold key insights. To tackle this issue, Retrieval-Augmented Generation (RAG), as introduced by Lewis *et al.* [10], retrieves relevant text chunks and leverages Large Language Models (LLMs) to enhance contextual understanding. RAG typically relies on dense vector indices of knowledge bases to provide context for generating responses, but can face challenges due to the finite context windows of LLMs and the restricted retrieval scope of RAG [7, 16].

Integrating knowledge graphs with Large Language Models (LLMs) can mitigate hallucinations and enhance performance in open-ended question answering. Liu *et al.* [12] combines pseudo-graph generation with atomic knowledge verification to query semantically encoded knowledge graphs. Microsoft's GraphRAG [4] demonstrates knowledge integration through advanced embedding techniques. GraphRAG's scalability and modularity are supported by community detection, which partitions graphs into contextually related clusters, enabling parallel summarisation at different levels of granularity, e.g., using the Leiden algorithm [15]. This clustering approach allows GraphRAG to overcome the challenges posed by limited context windows.

While syntactic proximity in knowledge graphs captures explicit connections, it can overlook latent semantic links [3], making it necessary to uncover and utilise indirect relationships. RankRAG addresses this by integrating context ranking with retrieval-augmented generation to prioritise the most relevant contextual inputs [19]. However, existing approaches such as RAG and RankRAG focus on direct retrieval and ranking, often failing to capture **latent semantic relationships** crucial for deeper document comparison. This limitation hinders the ability to extract **implicit contextual links** necessary for effective knowledge integration.

GraphMinds is a framework for contextual graph construction and analysis. The system takes user-uploaded documents, chunks the text, extracts nodes and edges using LLMs, and determines contextual proximity across chunks. Through transitive principles, GraphMinds constructs indirect relationships to uncover latent connections, enhancing both the structural richness of knowledge graphs and their utility for analytical tasks.

In summary, GraphMinds offers three key contributions:

- (1) A **novel approach** for constructing **contextual knowledge graphs** from text using LLMs, capturing both explicit and **latent semantic relationships**.
- (2) A methodology leveraging **transitive principles** to enhance the structural richness of extracted knowledge representations.
- (3) The application of **semantic distance metrics** to embeddings, enabling **community detection**.

We demonstrate that our approach enables **document comparison and retrieval** by uncovering **latent structures within knowledge graphs**, thereby, enhancing perspective identification.

2 Methodology

In this section, we explain the implementation of Latent Dirichlet Allocation that we have used to provide a comparison of topics identified from a set of recent journal papers that focus on a common theme, and the functioning of GraphMinds.

2.1 Selection of Papers

We have selected a set of seven journal papers that have been published recently on the subject of 'risks from artificial general intelligence'. Our aim is to compare perspective of the different authors, assuming that perspective identification can be made on the basis of the choice of topics included in the papers. The papers were selected because they provide a range of perspectives, from existential threat from AGI, to the need for stringent regulation of the technology, to checks and balances of AGI.

2.2 Latent Dirichlet Allocation

Latent Dirichlet Allocation is applied using libraries in R. The PDF of each paper is uploaded into R using `pdfutils`, and converted into a dataframe. We tokenize the dataframe (using `tidytext`) and remove stopwords (using the `stopwords` library). Next, using the `tm` library, we create a corpus for the document and convert the corpus into the Document Term Matrix. We then use the LDA function from the `topicmodels` library to extract topics (using the 'Gibbs' method). Using a value of 0.015 for beta, we extract terms with the highest weight for each topic and then manually label the topics, using as parsimonious a set of labels as possible.

2.3 GraphMinds

2.3.1 Data Preprocessing and Chunking. Uploading a PDF document is facilitated by the `PyPDFLoader` module from `LangChain`. After loading, the text is divided into chunks using the `RecursiveCharacterTextSplitter` class from `LangChain`. This method splits the text at natural boundaries, such as sentences or

paragraphs, to maintain contextual integrity while being of manageable size for processing.

- **Chunk Size:** Set to 1500 characters to balance readability and processing efficiency.
- **Chunk Overlap:** Defined as 150 characters. This overlap ensures continuity across chunks, preventing information loss when splitting occurs mid-sentence or at critical transitions in the text.
- **Length Function:** The `len()` function is used to calculate the length of text fragments, ensuring adherence to the pre-defined chunk size and overlap.

The examples in following sections use example sentences from a journal paper used in this study to illustrate the GraphMinds pipeline.

Input text:

- **Chunk 1:** *"The systematic review examined existing literature on Artificial General Intelligence (AGI), focusing on risk management, safety measures, and legal frameworks. The review followed PRISMA guidelines to ensure a comprehensive and transparent analysis of AGI-related research."*
- **Chunk 2:** *"...The review highlighted key risk categories associated with AGI, such as societal impacts, external containment strategies, and internal goal alignment mechanisms."*

The processed chunks are organised into a `DataFrame`. The columns include:

- **Text:** Contains the actual content of each chunk.
- **Source:** Indicates the PDF from which the text originated.
- **Page Number:** Specifies the page in the PDF from which the chunk was derived.
- **Chunk ID:** A unique identifier for each chunk to maintain traceability.

2.3.2 Concept Extraction and Categorisation. Following pre-processing, the next step extracts concepts from each text chunk using an LLM. We use the **Mistral 7B Model**, running locally via **Ollama** on an `nVidia GeForce RTX 4060 GPU`. A custom prompt guides the LLM in extracting and categorising key concepts while assigning contextual importance scores.

The process starts with a `DataFrame` (`df`), where each row represents a text chunk and its metadata, such as `chunk_id`. The function `df2Graph` applies the `graphPrompt` function to each row, passing the text and metadata as inputs. This call is executed using `apply()`, ensuring each row is processed individually by the model (`mistral-openorca:latest`). The result is a set of extracted concepts as JSON objects.

Prompt for Concept Extraction

"Your task is to extract the key concepts (and non-personal entities) mentioned in the given context. Extract only the most important and atomistic concepts; if needed, break the concepts down into simpler components. Categorise the concepts into one of the following categories: [event, concept, place, object, document, organisation, condition, misc]. Format your output as a list of JSON objects with the following structure:"

```
[
  {
    "entity": "TheConcept",
    "importance": "The contextual importance of the concept
on a scale of 1 to 5",
    "category": "TheTypeofConcept"
  },
  {...}
]
```

The model rates each extracted concept's contextual importance on a scale of 1 (Marginal or supplementary relevance) to 5 (Highly significant to the text's overall meaning to prioritise concepts for summarisation, indexing, or ontology creation).

Example of Importance Scoring

Input Text: "The systematic review examined existing literature on Artificial General Intelligence (AGI), focusing on risk management, safety measures, and legal frameworks."

Extracted Concepts:

```
1 [
2   {"entity": "systematic review", "category": "methodology", "
3     importance": 4},
4   {"entity": "Artificial General Intelligence", "category": "
5     technology", "importance": 5},
6   {"entity": "risk management", "category": "process", "importance"
7     : 5},
8   {"entity": "safety measures", "category": "policy", "importance":
9     4},
10  {"entity": "legal frameworks", "category": "regulation", "
11    importance": 4}
12 ]
```

The Graph-Based Relationship Extraction Prompt operates on the extracted concepts to identify connections based on co-occurrence in the same sentence or paragraph. The use of extracted concepts as nodes and defining edges based on contextual relationships, defines edges, between concepts, for the knowledge graph.

Prompt for Graph-Based Relationships

You are a network graph maker who extracts terms and their relations from a given context. You are provided with a context chunk. Your task is to extract the ontology of terms mentioned in the given context. These terms should represent the key concepts as per the context.

Thought 1: While traversing through each sentence, Think about the key terms mentioned in it. Terms may include object, entity, location, organization, person, condition, acronym, documents, service, concept, etc. Terms should be as atomistic as possible.

Thought 2: Think about how these terms can have one on one relation with other terms. Terms that are mentioned in the same sentence or the same paragraph are typically related to each other. Terms can be related to many other terms.

Thought 3: Find out the relation between each such related pair of terms.

Format your output as a list of json. Each element of the list contains a pair of terms and the relation between them, like the following:

```
[
  {
    "node_1": "A concept from extracted ontology",
    "node_2": "A related concept from extracted ontology",
    "edge": "relationship between the two concepts, node_1
and node_2 in one or two sentences"
  },
  {...}
]
```

Example of Nodes and their Connections:

Input Text: "The systematic review examined existing literature on Artificial General Intelligence (AGI), focusing on risk management, safety measures, and legal frameworks."

Graph-Based Output:

```
[
  {
    "node_1": "systematic review",
    "node_2": "Artificial General Intelligence",
    "edge": "examined research on"
  },
  {
    "node_1": "Artificial General Intelligence",
    "node_2": "risk management",
    "edge": "associated with"
  },
  {
    "node_1": "Artificial General Intelligence",
    "node_2": "safety measures",
    "edge": "requires"
  },
  {
    "node_1": "Artificial General Intelligence",
    "node_2": "legal frameworks",
    "edge": "regulated by"
  }
]
```

2.3.3 Relationship Mapping. The third stage of the process involves establishing both direct and indirect relationships between concepts to complete the knowledge graph. This step is critical as it not only identifies explicit connections but also uncovers hidden semantic links, enabling a richer representation of the data.

Direct Contextual Proximity. Direct contextual proximity measures the strength of the relationship between two nodes (Node 1 and Node 2) based on how frequently they appear together in text chunks. The more often a pair co-occurs within or across different chunks, the stronger their connection.

Processing Steps:

(1) Identifying Node Pairs

- Each pair of nodes (Node 1, Node 2) is extracted from the text along with the corresponding chunk IDs where they appear.
- (2) **Counting Co-occurrences**
- If a node pair appears multiple times within the same chunk, its weight increases proportionally to its frequency in that chunk.
 - If the same pair is found across multiple chunks, its weight continues to grow, and all the associated chunk IDs are collected into a list.
- (3) **Handling Self-loops**
- Self-loops, where a node is linked to itself (e.g., Node 1 → Node 1), are removed to ensure meaningful relationships between different nodes.

Direct Contextual Proximity is established when two concepts co-occur within or across different text chunks.

Input Text:

- **Chunk 1:** "The systematic review examined existing literature on Artificial General Intelligence (AGI), focusing on risk management, safety measures, and legal frameworks. The review followed PRISMA guidelines to ensure a comprehensive and transparent analysis of AGI-related research."
- **Chunk 2:** "...The review highlighted key risk categories associated with AGI, such as societal impacts, external containment strategies, and internal goal alignment mechanisms."

Extracted Relationships:

From Chunk 1:

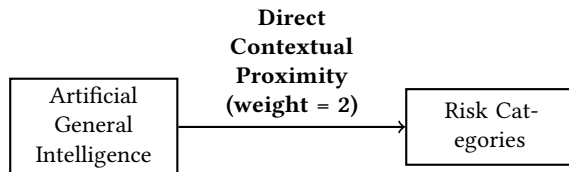
- Artificial General Intelligence → Risk Management
- Artificial General Intelligence → Safety Measures
- Artificial General Intelligence → Legal Frameworks
- Artificial General Intelligence → Risk Categories

From Chunk 2:

- Artificial General Intelligence → Risk Categories
- Risk Categories → Societal Impacts
- Risk Categories → External Containment Strategies
- Risk Categories → Internal Goal Alignment Mechanisms

Final Relationship Graph:

Since the pair (Artificial General Intelligence, Risk Categories) appears in both Chunk 1 and Chunk 2, its weight increases to 2. Below, only this relationship is shown in the final graph.



Indirect Contextual Proximity

Indirect relationships are inferred when two concepts do not co-occur within the same chunk but are linked through a shared intermediary. This creates additional edges in the graph and forms the core of the system's novelty.

- Identify all direct relationships and their associated chunks.

- For each concept, retrieve all other concepts directly connected to it (neighbours).
- Form pairs of these connected nodes, treating the current concept as the intermediary.
- Aggregate the chunk IDs involved in these indirect relationships to ensure traceability.

Extracted Relationships:

From Chunk 1:

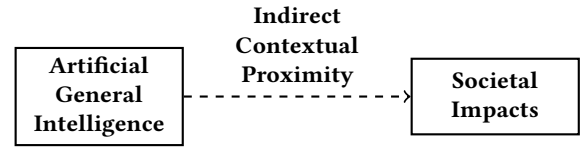
- Artificial General Intelligence → Risk Categories

From Chunk 2:

- Risk Categories → Societal Impacts

Final Indirect Connection:

Since **Artificial General Intelligence** is connected to **Risk Categories** in **Chunk 1** and **Risk Categories** is connected to **Societal Impacts** in **Chunk 2**, an **indirect contextual proximity** exists between **Artificial General Intelligence** and **Societal Impacts**.



To ensure efficient processing and avoid redundancy, the meta-data associated with each node pair is aggregated and filtered.

Avoiding Redundancy:

- For each node pair, merge all associated chunk IDs into a single list.
- Deduplicate the chunk IDs to ensure that no chunk is processed multiple times for the same relationship.
- Replace the chunk IDs with the corresponding text from the original dataset for interpretability and traceability.

Example:

Initial Metadata:

- Node 1: Artificial General Intelligence (AGI), Chunk IDs: [1, 3, 4, 6]
- Node 2: Risk Categories, Chunk IDs: [1, 2, 3, 4, 5]

After Deduplication:

- Combined Chunk IDs: [1, 2, 3, 4, 5, 6]

Rationale:

- By deduplicating chunk IDs, the system ensures that repeated processing of the same text is avoided, optimizing LLM usage and computational efficiency.

Irrelevant nodes and stopwords are removed from the dataset. In this paper, **named entities** are removed because the names of authors in citations as irrelevant nodes that do not offer semantic benefit to identifying the authors' perspectives.

Cleaning Process:

Named Entity Removal:

- Named entities are identified using SpaCy's large English model (en_core_web_lg) or multilingual model (xx_ent_wiki_sm).
- Nodes that match named entities are excluded from the cleaned dataset.

- **Stopword Removal:**

- Stopwords are identified using NLTK's English stopwords list.
- Nodes that match stopwords are also excluded.

Steps in Cleaning:

- For each node:
 - Check if it is a named entity using SpaCy.
 - Check if it is a stopwords using NLTK.
 - Retain nodes only if they are neither named entities nor stopwords.

Example:

- **Input Nodes:** ["Artificial General Intelligence", "safety", "the", "risks", "ethics", "Barrett & Baum"]
- **Filtered Nodes:** ["Artificial General Intelligence", "safety", "risks", "ethics"]

Updating the Dataset:

- After cleaning, nodes in the dataset are updated.
- Rows where all nodes are removed are discarded.

Output After Cleaning:

- Nodes are free from named entities and stopwords, ensuring the dataset remains focused on meaningful concepts.

2.3.4 Edge Refinement Using LLMs. Edges in the knowledge graph, initially labelled as "*direct contextual proximity*" or "*indirect contextual proximity*," undergo further refinement to ensure that relationships between nodes are descriptive and semantically precise. This step aims to replicate a human-like approach to knowledge graph construction, where connections are meaningful and clearly articulated. We use the **Mistral 7B Model** to refine edge labels.

- **Input Preparation:**

- For each pair of connected nodes (*Node 1* and *Node 2*), gather all associated text chunks.
- Concatenate these chunks into a unified text context for the LLM.

- **LLM Interaction:**

- Construct a prompt tailored to the task of labelling relationships. The prompt instructs the LLM to synthesise the provided context and generate a concise, meaningful label.

Prompt for Concept Extraction

Based on the following context, please provide a short and accurate label for the relationship between "Node 1" and "Node 2". The label should be concise and meaningful. Do not use any outside knowledge, only use the context provided.

Context: "Concatenated text chunks associated with Node 1 and Node 2."

Example:

- **Between Nodes:** "Artificial General Intelligence (AGI)" and "Societal Impacts"
- **Initial Label:** "indirect contextual proximity"
- **Refined Label:** "Artificial General Intelligence drives profound societal impacts, influencing economic, ethical, and governance structures."

- **Integration into the Dataset:**

- The initial proximity-based label is replaced with the refined, descriptive label. This ensures that the relationships are human-readable and semantically meaningful.

Final Dataset Structure: The final output combines direct and indirect edges, along with their associated metadata, into a comprehensive DataFrame, as shown in **Table 1**.

This structured approach ensures that relationships in the knowledge graph are both **interpretable** and **informative**.

2.3.5 Community Detection, Visualisation, and Local Knowledge Graph Generation. Community detection helps identify clusters of closely connected concepts within the knowledge graph. These communities reveal thematic groupings. The Girvan-Newman algorithm works by iteratively removing edges with the highest betweenness centrality, effectively breaking the graph into smaller subgraphs. This results in a hierarchical structure of communities.

Steps:

- **Graph Preparation:**

- Nodes in the graph represent concepts extracted from the dataset.
- Edges represent refined relationships between these concepts.
- Each edge is assigned a weight that reflects its contextual importance (e.g., derived from co-occurrence frequency).

- **Algorithm Application:**

- The Girvan-Newman algorithm is applied to the graph to iteratively detect communities.
- The first few levels of detected communities are extracted, representing the most cohesive clusters.

- **Community Formation:**

- Each detected community is a tightly connected group of nodes that share strong relationships.

To ensure meaningful representation of the communities:

Node Weighting:

- Nodes within a community are ranked based on their weighted degree (i.e., the sum of weights of all edges connected to a node).
- This highlights influential nodes within each community.

Sorting Nodes:

- Nodes in each community are sorted in descending order of their weighted degree, ensuring that the most influential nodes appear first.

Sorting Communities:

- Communities are sorted by size (number of nodes). As a tie-breaker, the average weight of nodes within each community is used.

The identified communities are visualised to provide insights into their structure and thematic boundaries:

- Each community is represented with a distinct colour, making the boundaries visually clear.
- The graph layout is optimised to position closely connected nodes near each other, ensuring the visualisation reflects the underlying structure.

Table 1: Refined Edge Labels for AGI Relationships

Node 1	Node 2	Previous Edge Label	Refined Edge Label	Chunk IDs	Weight
Artificial General Intelligence (AGI)	Risk Categories	Contextual Proximity	AGI plays a critical role in shaping risk categories for governance and safety policies.	[1,2]	2
Artificial General Intelligence (AGI)	Societal Impacts	Indirect Contextual Proximity	Artificial General Intelligence drives profound societal impacts, influencing economic, ethical, and governance structures.	[1,2]	1

Example Output Detected Communities:

- **Community 1:** ["Artificial General Intelligence", "risk management", "safety measures"]
- **Community 2:** ["ethical considerations", "governance frameworks"]

Visualisation Highlights:

- **Community 1:** Focuses on AGI risks and strategies for safety.
- **Community 2:** Revolves around ethical concerns and policy regulations.

The next step involves exporting the knowledge graph as an interactive HTML visualisation. Each community is assigned a unique colour and group identifier to visually distinguish it in the graph. Nodes belonging to the same community share the same colour, while the most influential node in each community (based on weighted degree) is highlighted.

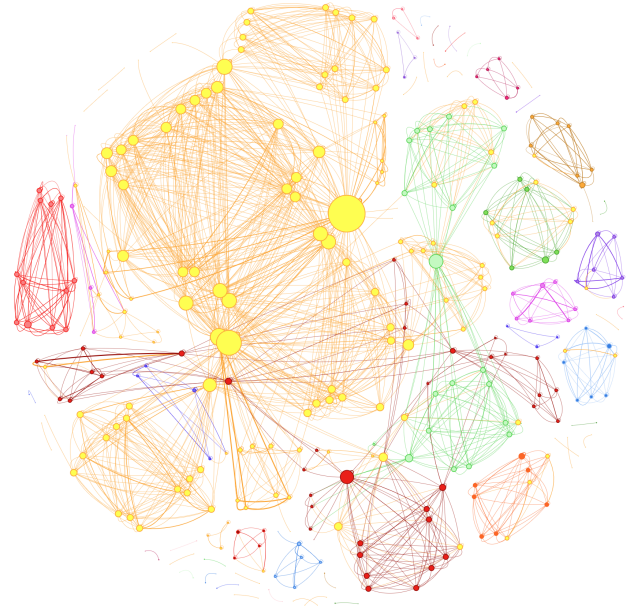
Steps:

- **Generate a Colour Palette:**
 - A shuffled colour palette is prepared, ensuring distinct colours for each community.
- **Community Data Assignment:**
 - For each community:
 - * Assign a colour and group ID.
 - * Identify the node with the highest weight (degree) in the community.
- **Node Attributes:**
 - Each node is assigned:
 - * **Group:** To determine its colour.
 - * **Colour:** Based on its community.
 - * **Size:** Proportional to its degree (connectivity).
 - * **Community Label:** Displayed as a tooltip for the central node.

An interactive graph visualisation is created using the Pyvis, to provide browser-based interactivity Figure 1.

Graph Features:

- **Node Representation:**
 - Nodes are sized based on their degree.
 - Central nodes of communities are highlighted with a distinct golden colour and larger size.
- **Edge Representation:**
 - Edges are weighted by their contextual importance, influencing their thickness in the visualisation.
 - Edge labels display the refined relationship between connected nodes.

**Figure 1: Automatically generated knowledge graph. The graph displays multiple communities with distinct colours.****Interactivity:**

- Users can select nodes to view additional details.
- Physics-based layout ensures the graph is visually appealing and easy to interpret.

Interactive Features:

- Hovering over a node or edge reveals additional metadata, such as relationships or community labels, as shown in Figure 2.
- Physics settings allow nodes to dynamically rearrange for clarity.

2.3.6 Embedding Generation and Similarity Matching. To enable intelligent and semantically accurate query-response interactions with the knowledge graph, node-edge-node triples are transformed into dense vector embeddings. These embeddings facilitate efficient matching between user queries and graph relationships, allowing for meaningful retrieval of relevant information and context.

Objective: Convert graph relationships into dense vector embeddings that effectively capture their semantic meaning.

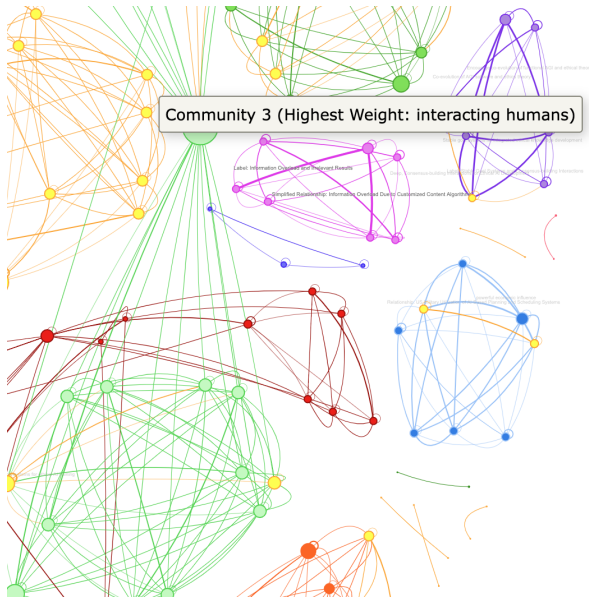


Figure 2: Hover interaction highlighting a central node. The tooltip provides information about the most influential node in a specific community.

Node-Edge-Node Triplets:

- Each relationship in the knowledge graph is represented as a structured triplet consisting of two nodes connected by an edge.
- Example:**
 - Node 1:** Artificial General Intelligence (AGI)
 - Edge:** AGI plays a critical role in shaping risk categories for governance and safety policies.
 - Node 2:** Risk Categories
- Triplet Representation:** Artificial General Intelligence (AGI) → AGI plays a critical role in shaping risk categories for governance and safety policies → Risk Categories

Embedding Generation:

- The all-MiniLM-L6-v2 model from SentenceTransformers is used to generate dense vector embeddings for these triplets.
- This model captures semantic nuances, enabling precise alignment with natural language queries.

Objective: Embed user queries into the same vector space as the graph relationships for effective retrieval.

Query Preparation:

- User queries are preprocessed to maintain consistency in formatting and ensure compatibility with the embedding model.
- Example Query:** "How does AGI influence risk categorisation in governance?"

Embedding Process:

- The same SentenceTransformer model is utilized to embed the user query into the vector space.

- The resulting query vector is compared with the graph relationship embeddings using cosine similarity.

Similarity Matching Objective: Identify the most relevant graph relationships by computing their semantic similarity with the user query.

Cosine Similarity Computation:

- Compute cosine similarity between the embedded user query and the precomputed embeddings of graph relationships.
- Higher similarity scores indicate stronger semantic relevance.

Identifying Relevant Matches:

- Retrieve all relationships with a similarity score of at least 0.5.
- The most relevant match is passed to the LLM for response generation.

Example:

- User Query:** "How does AGI influence risk categorization in governance?"
- Graph Relationship Match (Similarity ≥ 0.5):**
 - Node 1:** Artificial General Intelligence (AGI)
 - Edge:** AGI plays a critical role in shaping risk categories for governance and safety policies.
 - Node 2:** Risk Categories
 - Similarity Score:** 0.92 (high similarity)

Processing the Matched Relationship:

- Since the similarity score (0.92) is above the 0.5 threshold, the triplet is selected for response generation.
- The matched relationship is passed to the LLM to construct a natural language answer.

Generated Response:

- LLM Output:** "Artificial General Intelligence (AGI) plays a critical role in shaping risk categories for governance and safety policies. By analyzing various risk factors, AGI helps policy-makers create frameworks that mitigate potential threats while ensuring responsible implementation."

3 Results

From the selected journal papers, we extract a set of topics using Latent Dirichlet Allocation (LDA) and identify clusters of communities from the GraphMinds Knowledge Graphs. The following sections provide a detailed description of these topics and clusters.

3.1 Semantic Similarity of Output from LDA

For each paper, a set of topics were identified using LDA. Each topic was labelled manually, with an effort to minimise the number of labels so they could apply to multiple papers. The goal was to cluster papers by common topics.

Using these topic labels, semantic similarity scoring was applied to generate a cluster graph. As Figure 3 shows, despite common labelling, papers rarely clustered. Instead, each paper contained a unique set of topics reflecting the authors' perspectives, with limited overlap—an unexpected outcome given the shared subject.

At the top of the image, represented with a dotted red line cluster, several nodes relate to goal alignment and AI bias. A specific

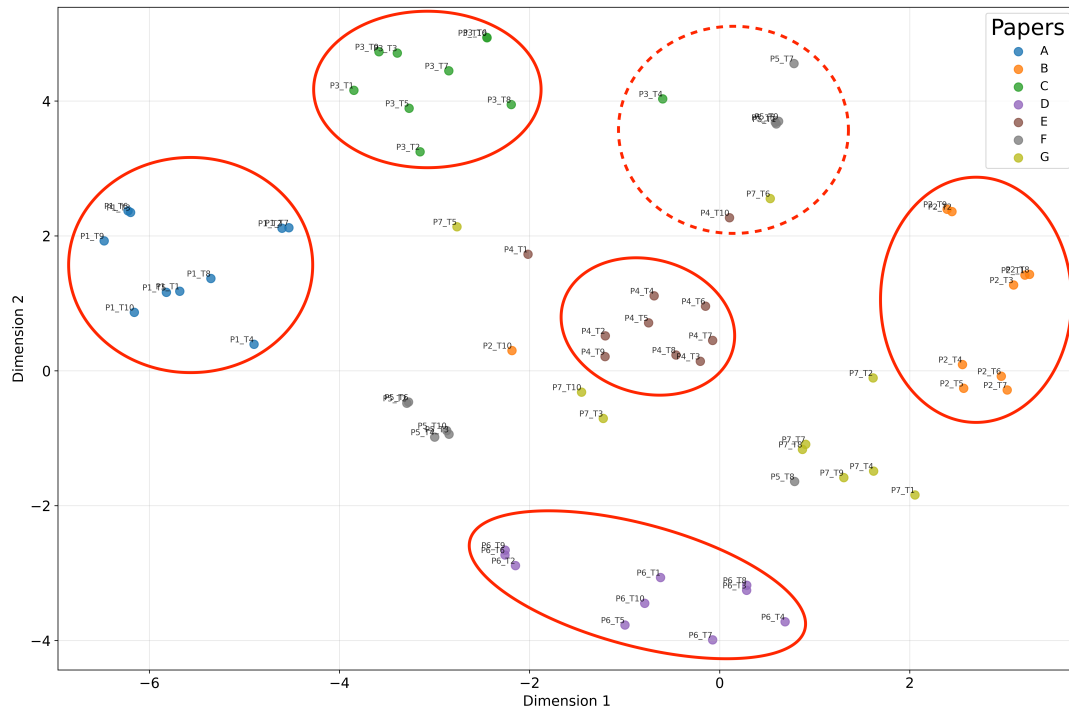


Figure 3: Visualization of LDA-derived topic embeddings, coloured by paper source and clustered by similarity.: (A) [14], (B) [1], (C) [17], (D) [8], (E) [2], (F) [5], (G) [13].

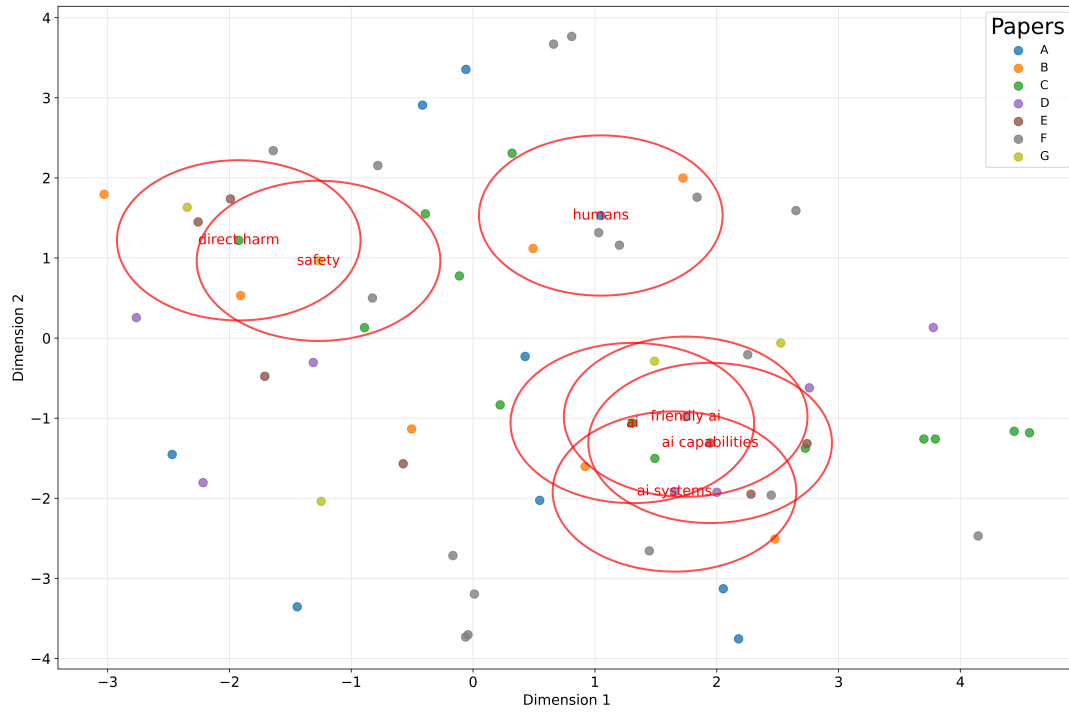


Figure 4: Clusters of communities extracted from the GraphMinds Knowledge Graph, with each dot representing a central node. Red ellipses highlight key thematic groupings.: (A) [14], (B) [1], (C) [17], (D) [8], (E) [2], (F) [5], (G) [13].

node, **P3_T4**, represents *Paper 3 (Leakproofing Singularity: Artificial Intelligence Confinement Problem)*, which discusses goal alignment. Similarly, **P4_T10**, **P7_T16**, and **P5_T1** also address goal alignment. These correspond to the papers *Knowledge Graph Refinement: A Survey of Approaches and Evaluation Methods (P4)*, *The Risks Associated with Artificial General Intelligence: A Systematic Review (P7)*, and *Nine Ways to Bias Open-Source AGI Toward Friendliness (P5)*.

This suggests that while themes such as goal alignment and AI bias appear across multiple papers, their framing varies significantly, limiting clustering.

3.2 Semantic Similarity of Output from GraphMinds

We assume that a 'topic' defined by LDA is conceptually similar to the cluster of communities identified from GraphMinds knowledge graphs. Both methods provide high-level groupings of concepts that can be used for semantic distance calculations. However, their outputs differ in structure and distribution.

Figure 4 suggests that GraphMinds clusters tend to incorporate concepts from multiple papers, whereas LDA primarily groups concepts within individual papers. This contrast highlights differences in how the two methods capture conceptual relationships, with GraphMinds reflecting broader cross-paper linkages.

Note: In the visualisation, nodes are coloured according to their source paper, making key concepts easily distinguishable. Additionally, detected communities are marked with **red boundary** circles around closely connected nodes to emphasize their relationships.

3.3 Comparison of Results

Table 2 reports cosine similarity between LDA topics and GraphMinds communities. The cosine similarity scores indicate thematic connections between AI-related concepts, suggesting some agreement between topic modeling and graph-based clustering. We set a threshold of 0.4 similarity for consideration. AI systems and capabilities with AI System (0.6), Evaluation and Checks with Human Evaluation (0.45), Computation System with AI System (0.47), Alignment with Human Goals and Alignment of Human and AI Values (0.41) reflect common sub-themes identified by both methods.

It is interesting to note that the strongest similarity (0.7) was between AI systems and capabilities (for LDA) with Large Scale Social Harms (for GraphMinds). We hypothesise that GraphMinds was able to uncover this link by identifying connections between discussions on AI capabilities and concerns about risks arising from the use of advanced AI capabilities. These risks were discussed in the papers in relation to policy debates, ethical AI research, and real-world case studies, and suggest that advanced AI is frequently discussed in the context of potential harms.

These findings indicate that LDA and GraphMinds independently capture similar conceptual structures, reinforcing the robustness of AI discourse analysis through different methodologies.

4 Conclusion

By leveraging embeddings from the knowledge graph, *GraphMinds* uncovers latent semantic relationships between nodes in a knowledge graph, even when they are separated by multiple edges. These

Table 2: LDA Topics and GraphMinds Central Nodes with Cosine Similarity

LDA Topic		GraphMinds Central Node	Cosine Similarity
Evaluation and Checks		Human Evaluation	0.45
AI systems and capabilities		Large Scale Social Harms	0.7
AI systems and capabilities		AI-based Planning and Scheduling System	0.41
AI systems and capabilities		AI System	0.6
Computation System		AI System	0.47
Alignment with Human Goals		Alignment of Human and AI Values	0.41

relationships allow us identify communities (i.e., clusters of interconnected nodes) that serve as topic groupings. The central node within each community acts as the topic label. We then compute semantic distances on these labels, facilitating an intuitive and structured visualization of document similarities.

For perspective identification, the topic clusters derived through Latent Dirichlet Allocation (LDA) (Figure 3) indicate how the papers exhibit distinct perspectives, with limited thematic overlap. The only notable exception is *goal alignment*, which appears across multiple papers. In contrast, GraphMinds (Figure 4) reveals broader thematic connections across papers, particularly regarding AI capabilities and behavior, direct harm and safety, and human impact. This suggests that GraphMinds uncovers shared perspectives across diverse sources.

GraphMinds also identifies communities that are specific to individual papers, e.g., in Figure 4 four nodes from Paper F in the lower center and four nodes from Paper C in the lower right. Unlike LDA, which primarily captures intra-document topic structures, GraphMinds facilitates a comparative analysis of perspectives across different documents by identifying both proximal and distal connections within the knowledge graph. This illustrates GraphMinds' **contextual richness** as it is retrieving not only directly related text chunks but also those that are semantically relevant yet distant in the knowledge graph. The ability to map thematic relationships across disparate sources could be valuable for exploratory research and large-scale literature analysis. By enabling both explicit and latent relationship mapping, GraphMinds uncovers deep semantic structuring of knowledge graphs, which could improve their utility for policy analysis, and decision support. Future work will focus on refining the integration of external knowledge sources, optimizing scalability for large-scale datasets, improving real-time retrieval capabilities for user interaction, and exploring the opportunities for application in different user communities and domains.

References

- [1] Yoshua Bengio. 2024. Government interventions to avert future catastrophic AI risks. *Harvard Data Science Review* Special Issue 5 (2024).
- [2] Yoshua Bengio, Geoffrey Hinton, Andrew Yao, Dawn Song, Pieter Abbeel, Trevor Darrell, Yuval Noah Harari, Ya-Qin Zhang, Lan Xue, Shai Shalev-Shwartz, et al. 2024. Managing extreme AI risks amid rapid progress. *Science* 384, 6698 (2024), 842–845.
- [3] Philipp Cimiano and Heiko Paulheim. 2017. Knowledge graph refinement: A survey of approaches and evaluation methods. *Semant. Web* 8, 3 (jan 2017), 489–508. <https://doi.org/10.3233/SW-160218>
- [4] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130 [cs.CL] <https://arxiv.org/abs/2404.16130>
- [5] Ben Goertzel and Joel Pitt. 2012. Nine ways to bias open-source AGI toward friendliness. *Journal of Evolution and Technology* 22, 1 (2012), 116–131.
- [6] Manuela Gómez-Suta, Julián Echeverry-Correa, and José A. Soto-Mejía. 2023. Stance detection in tweets: A topic modeling approach supporting explainability. *Expert Syst. Appl.* 214, C (March 2023), 22 pages. <https://doi.org/10.1016/j.eswa.2022.119046>
- [7] Travis Goodwin, Max Savery, and Dina Demner-Fushman. 2020. Flight of the PEGASUS? Comparing Transformers on Few-shot and Zero-shot Multi-document Abstractive Summarization. In *Proceedings of the 28th International Conference on Computational Linguistics*, Donia Scott, Nuria Bel, and Chengqing Zong (Eds.). International Committee on Computational Linguistics, Barcelona, Spain (Online), 5640–5646. <https://doi.org/10.18653/v1/2020.coling-main.494>
- [8] BORIS D Grozdanoff. 2023. The Looming Shadow: Taking Seriously Potential Existential Threats Brought about by Artificial Intelligence. *Ethical Studies* (2023), 100–106.
- [9] Dilek Küçük and Fazli Can. 2020. Stance Detection: A Survey. *ACM Comput. Surv.* 53, 1, Article 12 (Feb. 2020), 37 pages. <https://doi.org/10.1145/3369026>
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [11] Wei-Hao Lin, Theresa Wilson, Janyce Wiebe, and Alexander Hauptmann. 2006. Which side are you on? identifying perspectives at the document and sentence levels. In *Proceedings of the Tenth Conference on Computational Natural Language Learning* (New York City, New York) (CoNLL-X '06). Association for Computational Linguistics, USA, 109–116.
- [12] Jiaxiang Liu, Tong Zhou, Yubo Chen, Kang Liu, and Jun Zhao. 2024. Enhancing Large Language Models with Pseudo- and Multisource- Knowledge Graphs for Open-ended Question Answering. arXiv:2402.09911 [cs.CL] <https://arxiv.org/abs/2402.09911>
- [13] Scott McLean, Gemma JM Read, Jason Thompson, Chris Baber, Neville A Stanton, and Paul M Salmon. 2023. The risks associated with Artificial General Intelligence: A systematic review. *Journal of Experimental & Theoretical Artificial Intelligence* 35, 5 (2023), 649–663.
- [14] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983 [cs.CL] <https://arxiv.org/abs/2311.12983>
- [15] V. A. Traag, L. Waltman, and N. J. van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* 9, 1 (March 2019). <https://doi.org/10.1038/s41598-019-41695-z>
- [16] Alex Warstadt, Yu Cao, Ioana Grosu, Wei Peng, Hagen Blix, Yining Nie, Anna Alsop, Shikha Bordia, Haokun Liu, Alicia Parrish, Sheng-Fu Wang, Jason Phang, Anhad Mohananey, Phu Mon Htut, Paloma Jeretic, and Samuel R. Bowman. 2019. Investigating BERT's Knowledge of Language: Five Analysis Methods with NPIs. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 2877–2887. <https://doi.org/10.18653/v1/D19-1286>
- [17] Roman V Yampolskiy. 2012. Leakproofing singularity-artificial intelligence confinement problem. *Journal of Consciousness Studies* JCS (2012).
- [18] Jian Yang, Xinyu Hu, Gang Xiao, and Yulong Shen. 2024. A Survey of Knowledge Enhanced Pre-trained Language Models. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* (mar 2024). <https://doi.org/10.1145/3631392> Just Accepted.
- [19] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. 2024. RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs. arXiv:2407.02485 [cs.CL] <https://arxiv.org/abs/2407.02485>